NYC Yellow Taxi Demand-Supply Allocation & Simulation Framework

Technical Reference Manual & System Architecture Guide

Author: Data Engineering & Analytics Lead | Target Audience: Evaluation Committee & Technical Reviewers
System Environment: Databricks, PySpark, Delta Lake, Streamlit | Dataset: NYC TLC Yellow Taxi (8.7M Trips)

สารบัญและโครงสร้างเอกสาร (Table of Contents)

1. Executive Summary & Problem Statement (ภาพรวมผู้บริหารและที่มาของปัญหา)

1.1 ที่มาของปัญหา Demand-Supply Imbalance ใน NYC Yellow Taxi

ระบบขนส่งสาธารณะด้วยรถแท็กซี่สีเหลือง (Yellow Taxi) ในนครนิวยอร์ก (NYC) เผชิญกับปัญหาเชิงโครงสร้างที่สำคัญคือ ความไม่สมดุลระหว่างความต้องการใช้บริการของผู้โดยสาร (Passenger Demand) และปริมาณรถแท็กซี่ที่พร้อมให้บริการ (Taxi Supply) ในมิติของพื้นที่ (Spatial Dimension) และช่วงเวลา (Temporal Dimension) ส่งผลให้เกิดภาวะแท็กซี่ขาดแคลนขั้นวิกฤตในบางเขต ในขณะที่เขตข้างเคียงกลับมีรถแท็กซี่จอดว่างหรือขับวนรอผู้โดยสาร (Cruising / Deadheading) อย่างเปล่าประโยชน์

1.2 สรุปผลการวิเคราะห์สภาวะไม่สมดุลจากข้อมูลจริง (Jan 2018 Data Insights)

จากการประมวลผลข้อมูลการเดินทางจริงของ NYC TLC Yellow Taxi จำนวน 8.7 ล้านรายการ ในเดือนมกราคม 2018 โดยใช้อัตราส่วน Demand-Supply Ratio (คำนวณจาก Pickup Count / Dropoff Count) สามารถสรุปสภาวะความไม่สมดุลระดับเขต (Borough Level) ได้ดังนี้:

เขตพื้นที่ (Borough)	Demand (Pickups)	Supply (Dropoffs)	Ratio	สถานะระบบ (Status)	รายละเอียดการกระจายตัว
Queens	515,969	379,809	1.36	ขาดแคลน (Shortage)	ขาดแคลนรวม ~136,000 ทริป/เดือน พุ่งสูง 1.71 ช่วง Evening Rush
Brooklyn	98,387	355,507	0.28	ล้นระบบ (Overflow)	รถเกินความต้องการ ~257,000 ทริป/เดือน ขับวนว่างสูงมาก

Bronx	6,653	50,181	0.13	ล้นระบบ (Overflow)	Supply มากกว่า Demand ถึง 7.5 เท่า (รถมาส่งแล้วขับกลับว่าง)
Manhattan	7,900,000	7,800,000	1.02	สมดุล (Balanced)	ความต้องการและรถพร้อมให้บริการอยู่ในภาวะใกล้เคียงกัน
Staten Island	108	1,800	0.06	ล้นระบบ (Overflow)	ปริมาณการเรียกใช้น้อยมาก การเดินทางส่วนใหญ่เป็น Dropoff
EWR (Newark)	167	5,500	0.03	ล้นระบบ (Overflow)	เป็นปลายทางส่งผู้โดยสารข้ามรัฐ ไม่มีจุดรับคืนในเมือง

1.3 ตัวเร่งวิกฤต (Crisis Catalysts): โซนสนามบิน JFK และ LaGuardia

เมื่อเจาะลึกวิเคราะห์ในระดับโซนย่อย (Zone Level) พบว่าสาเหตุหลักที่ทำให้ Queens เกิดภาวะขาดแคลนแท็กซี่รุนแรง มาจากโซนสนามบินหลัก 2 แห่ง ได้แก่ JFK Airport (Demand-Supply Ratio สูงถึง 3.35) และ LaGuardia Airport (Ratio สูงถึง 2.82) เนื่องจากผู้โดยสารสายการบินมาถึงเป็นรอบ (Flight Arrival Batches) ทำให้เกิด Demand Spike มหาศาล ในขณะที่พื้นที่ท้องถิ่นอื่นใน Queens (เช่น Astoria, Long Island City) มีอัตราส่วนใกล้เคียงภาวะสมดุล การแก้ปัญหาจึงต้องใช้กลยุทธ์เฉพาะพื้นที่สนามบิน

1.4 ประเมินมูลค่ารายได้ที่กู้คืนได้หลังมีระบบ (Revenue Recovered Valuation)

การประมาณการมูลค่ารายได้ที่สามารถกู้คืนกลับมาได้ในระบบ (Revenue Recovered) เมื่อมีการจัดสรรรถแท็กซี่อย่างมีประสิทธิภาพในโซนที่ขาดแคลน จำเป็นต้องแยกความแตกต่างระหว่างจำนวนเที่ยววิ่ง (Trips / Dropoff Count) ที่ปรากฏในแอปพลิเคชัน (เช่น 25,675 เที่ยววิ่งต่อช่วง peak hour) กับจำนวนตัวรถแท็กซี่จริง (Physical Medallion Taxis) ซึ่งมีประมาณ 13,500 คันทั่ว NYC โดยแต่ละคันวิ่งเฉลี่ยประมาณ 2 เที่ยวต่อชั่วโมง

- ต่อช่วงเวลาเร่งด่วน (Peak Hour): ~$573,000 / ชั่วโมง (คำนวณจากเที่ยววิ่งแท็กซี่ที่จัดสรรใหม่ได้จริง ประมาณ 7,000 ทริป)
- ต่อวัน (Daily Aggregate): ~$3.9 Million / วัน
- ต่อปี (Annual Projection - Realistic Estimate): $500M – $800M / ปี (คิดเป็นประมาณ 3.3% – 5.3% ของรายได้รวมภาคธุรกิจแท็กซี่ใน NYC)

> 📌 **MACROECONOMIC VALUATION NOTE**
> การประเมินมูลค่าระดับมหภาคข้างต้น มีการแยกบริบทระหว่างจำนวนเที่ยววิ่ง (Trips) และจำนวนรถแท็กซี่จริง (Vehicles) อย่างชัดเจน ตัวเลขประมาณการสมจริงที่ $500M–$800M/ปี สะท้อนถึงศักยภาพการจัดสรรรถจริงภายใต้ข้อจำกัดด้านจำนวนใบอนุญาต (NYC Medallion Cap ~13,500 คัน) เพื่อความโปร่งใสและถูกต้องตามหลักวิศวกรรมข้อมูล

2. Data Pipeline & Engineering Architecture (สถาปัตยกรรมข้อมูลและการประมวลผล)

2.1 สถาปัตยกรรมระบบ Big Data และ Data Pipeline เต็มรูปแบบ

ระบบประมวลผลข้อมูลได้รับการออกแบบบนสถาปัตยกรรม Big Data เพื่อรองรับการวิเคราะห์และเชื่อมโยงข้อมูลการเดินทางอย่างครบวงจร โดยมีสายโซ่การประมวลผลข้อมูล (Data Pipeline Chain) ตั้งแต่ต้นน้ำถึงปลายน้ำ ดังนี้:

1. ข้อมูลดิบ (Raw Data Tier): นำเข้าข้อมูล NYC TLC Yellow Taxi Trip Records ย้อนหลังปี 2015–2018 (โดยเฉพาะมกราคม 2018 มีปริมาณกว่า 8.7 ล้านทริป) ซึ่งประกอบด้วยฟิลด์สำคัญ ได้แก่ tpep_pickup_datetime, tpep_dropoff_datetime, PULocationID, DOLocationID, trip_distance, fare_amount, tip_amount, total_amount และ RatecodeID

2. กระบวนการ ETL/ELT (NYC TLC CSV to Delta Layer): ทำการกรองและทำความสะอาดข้อมูล (Data Cleaning) แปลงฟิลด์ datetime สกัดคุณลักษณะเวลา ทำการแมปข้อมูล LocationID เข้ากับ Borough และ Zone Name ผ่าน taxi_zone_lookup.csv พร้อมสรุปรวบยอดข้อมูล Demand-Supply ในระดับ Zone × Hour

3. การกำหนดนิยาม Demand & Supply Framework: กำหนดให้ Demand วัดจากปริมาณการรับผู้โดยสาร (Pickup Count) ในโซนนั้นๆ ส่วน Supply ใช้ปริมาณการส่งผู้โดยสาร (Dropoff Count) เป็น Proxy ตัวแทนรถแท็กซี่ที่พึ่งส่งผู้โดยสารเสร็จและพร้อมรับงานต่อ โดยมีข้อจำกัดที่รับทราบคือ ไม่รวมสถานะรถว่างจริง (Cruising) หรือรถที่จอดพักงาน

ตัวอย่าง

กรณีที่ 1: Spatial mismatch (ต่างที่)

คนขึ้นแท็กซี่ที่ **Manhattan** (demand เพิ่ม) ไปลงที่ **Brooklyn** (supply เพิ่ม) → Manhattan ขาดรถ, Brooklyn ล้นรถ — ทั้งที่เป็นคนเดียวกัน 1 trip

กรณีที่ 2: Temporal mismatch (ต่างเวลา)

คนขึ้นแท็กซี่เวลา 17:55 ที่ **Penn Station** ชั่วโมงที่ 17 ไปลงเวลา 18:10 ที่ **JFK** ชั่วโมงที่ 18→ demand นับในชั่วโมง 17, supply นับในชั่วโมง 18

กรณีที่ 3: ทั้งคู่

คนขึ้นที่ Midtown 17:50 ลงที่ JFK 18:15 → demand เพิ่มที่ Midtown ชั่วโมงที่ 17, supply เพิ่มที่ JFK ชั่วโมงที่ 18 — ต่างทั้ง zone และ hour

คำ	นับอะไร	จากฟิลด์ไหน	เมื่อไหร่
Demand	จำนวนคนเรียกแท็กซี่ (pickup)	tpep_pickup_datetime + PULocationID	ที่ zone ต้นทาง, ตามเวลาที่ขึ้นรถ
Supply	จำนวนแท็กซี่มาส่งผู้โดยสาร (dropoff)	tpep_dropoff_datetime + DOLocationID	ที่ zone ปลายทาง, ตามเวลาที่ลงรถ

4. แอพพลิเคชันและการเสิร์ฟข้อมูล (Streamlit Serving Layer): นำส่งข้อมูลที่ผ่านการประมวลผลและปรับแต่งแล้วในรูปแบบไฟล์ parquet ประสิทธิภาพสูง (zone_hourly_enhanced.parquet) เพื่อใช้เป็นฐานข้อมูลหลักในการประมวลผลใน Streamlit Web Application

5. กลไกการจำลองและประมวลผลในแอปพลิเคชัน (App Execution & Allocation Simulation): ฟังก์ชัน apply_scenario ประมวลผลจำลองการย้ายทริปแท็กซี่จากพื้นที่ล้นระบบ (Overflow Zones) ไปยังพื้นที่ขาดแคลน (Shortage Zones) พร้อมคำนวณผลประโยชน์สุทธิทางการเงิน (Cost-Benefit Analysis) และอัปเดตสภาวะความสมดุลใหม่แบบ Real-time

2.2 การแก้ปัญหา Schema Migration และ Geocoding Alignment ข้ามยุค

ข้อจำกัดสำคัญของชุดข้อมูล NYC TLC คือการเปลี่ยนแปลงโครงสร้างจัดเก็บพิกัดข้ามปี กล่าวคือ ในช่วงปี 2015–2016 ข้อมูลบันทึกเป็นพิกัด GPS (Latitude/Longitude) แต่ปี 2017–2018 เปลี่ยนมาใช้ Taxi Zone LocationID (1–263) ระบบจึงสถาปนาสถาปัตยกรรม Data Mapping เพื่อเชื่อมโยงข้อมูลดังนี้:

1. Ingestion & Join Layer: ใช้ taxi_zone_lookup.csv เพื่อแมป PULocationID และ DOLocationID เข้ากับชื่อเขต (Borough) และโซน (Zone Name)
2. Geospatial Alignment: ใช้ไฟล์ NYC Taxi Zones GeoJSON (NYC_GeoJson.geojson) ในการหาจุดกึ่งกลางโซน (Zone Centroids) และแปลงพิกัด GPS ข้อมูลปีเก่าให้เข้ากับระบบ LocationID
3. Unified Schema Standard: แปลงข้อมูลทุกชุดให้อยู่ในมาตรฐานเดียวกันเพื่อทำ Time Series Analysis ย้อนหลังได้อย่างต่อเนื่อง

2.3 เหตุผลในการเลือกใช้ข้อมูลปี 2018 สำหรับการวิเคราะห์ระดับ Zone เทียบกับข้อมูลปี 2015–2018 สำหรับ Trends และ Forecasting

การเลือกระยะเวลาของชุดข้อมูลในแต่ละส่วนของการวิเคราะห์ ได้รับการออกแบบให้สอดคล้องกับข้อจำกัดทางเทคนิคและวัตถุประสงค์ของการใช้งาน ดังนี้:

1. การใช้ข้อมูลปี 2018 สำหรับ Zone-Level Allocation Analysis: ข้อมูลการเดินทางของ NYC TLC มีการเปลี่ยนระบบระบุพิกัดจาก GPS Coordinates (Latitude/Longitude) มาเป็น Taxi Zone LocationID (1–263) ตั้งแต่ช่วงปี 2017 เป็นต้นมา การใช้ข้อมูลปี 2018 ซึ่งเป็นปีล่าสุดในชุดข้อมูล จึงช่วยให้การวิเคราะห์โครงสร้าง Demand-Supply ในระดับ Zone มีความแม่นยำ สะท้อนพฤติกรรมการเดินทางและสภาพแวดล้อมที่เป็นปัจจุบันที่สุด โดยไม่ต้องเผชิญความคลาดเคลื่อนจากการแปลงพิกัด GPS ย้อนหลัง

2. การใช้ข้อมูลปี 2015–2018 สำหรับ Macro Trends & Forecasting: การศึกษาภาพรวมแนวโน้มการเดินทางในระดับมหาภาค (Macro Level) และการวิเคราะห์อนุกรมเวลา (Time Series Analysis) จำเป็นต้องอาศัยข้อมูลครอบคลุมหลายช่วงเวลา เพื่อให้มองเห็นรูปแบบการเปลี่ยนแปลง (Historical Trends) และพฤติกรรมตามฤดูกาลในภาพใหญ่

3. การรับมือกับข้อจำกัดข้อมูลเฉพาะเดือนมกราคม (January-Only Limitations): แม้ชุดข้อมูลทดสอบจะมีเฉพาะเดือนมกราคม แต่พฤติกรรมความต้องการเดินทางในช่วง Rush Hour รายวันยังคงมีรูปแบบ (Pattern) ที่ชัดเจนและเป็นตัวแทนที่ดีสำหรับการสร้างแบบจำลองการจัดสรรรถ (Allocation Model) ทั้งนี้ ในแผนการพัฒนาระยะถัดไป จะขยายการดึงข้อมูลให้ครอบคลุมทั้ง 12 เดือน เพื่อให้ครอบคลุม Seasonal Pattern ครบถ้วนตลอดทั้งปี

2.4 บทบาทของคอลัมน์ข้อมูล (Feature Engineering Spectrum)

ชื่อคอลัมน์ (Field)	ประเภทข้อมูล	บทบาทและฟังก์ชันในระบบ	ขั้นตอนที่นำไปใช้งาน
tpep_pickup_datetime	Timestamp	คำนวณ Passenger Demand (Pickup Count) รายชั่วโมง	Demand Analysis & Forecasting
tpep_dropoff_datetime	Timestamp	คำนวณ Taxi Supply Proxy (Dropoff Count) และ Trip Duration	Supply Analysis & Cost Validation
PULocationID	Integer	ระบุโซนต้นทางของการเรียกใช้บริการ (Pickup Zone ID)	Spatial Demand Aggregation
DOLocationID	Integer	ระบุโซนปลายทางของการส่งผู้โดยสาร (Dropoff Zone ID)	Spatial Supply Aggregation
trip_distance	Float	ตรวจสอบระยะทางจริงเทียบกับระยะทางตรง (Euclidean Distance)	Cost & Travel Time Validation
tip_amount	Float	สร้าง Tip Map แสดงอัตราทิปเฉลี่ยรายโซนเพื่อเป็น Market Incentive	Incentive Mechanism Design
RatecodeID	Integer	จำแนกประเภทการเดินทาง (1=Standard, 2=JFK Flat Rate $52)	Airport vs Local Strategy Classification
fare_amount / total_amount	Float	คำนวณรายได้ต่อเที่ยวและประเมินมูลค่าความคุ้มค่าทางเศรษฐกิจ	Cost-Benefit & Revenue Analysis

3. Mathematical Formulation & Physical Constraints (สูตรคณิตศาสตร์และเงื่อนไขทางกายภาพ)

3.1 การกำหนดตัวแปรและสมการอัตราส่วน Demand-Supply Ratio

การวัดสภาวะความสมดุลใช้ตัวแปร Demand-Supply Ratio R_i,t ของโซน i ในช่วงเวลา t คำนวณจาก:

R_{i,t} = Demand_{i,t} / Supply_{i,t}
โดยที่ Demand_{i,t} คือจำนวน Pickup และ Supply_{i,t} คือจำนวน Dropoff (Proxy) ในโซนนั้นๆ

3.2 ช่วงสมดุล (Equilibrium Thresholds) และเหตุผลของ 20% Safety Buffer

การคำนวณช่วงสมดุลและ Safety Buffer 20%

1. เกณฑ์ตัดสินความสมดุล (Ratio Threshold):

- กำหนดช่วงสมดุลไว้ที่ Ratio 0.8 ถึง 1.2 (ยอมรับความผันผวนบวกลบ 20%)
- Ratio > 1.2 → แท็กซี่ขาดแคลน (Shortage Zone)
- Ratio < 0.8 → แท็กซี่ล้นระบบ (Overflow Zone)
- Ratio 0.8–1.2 → สภาวะสมดุล (Balanced Zone)

2. สมการคำนวณปริมาณรถจัดสรร (Safety Buffer 20%):

- ปริมาณรถส่วนเกินที่ย้ายออกได้ (Excess) = (จำนวนรถที่มี × 80%) - ความต้องการผู้โดยสาร
  (สำรองรถไว้ 80% ในพื้นที่เดิมเสมอ เพื่อไม่ให้เขตต้นทางกลายเป็นโซนขาดแคลนเสียเอง)

- ปริมาณรถที่ต้องการเติม (Shortfall) = ความต้องการผู้โดยสาร - (จำนวนรถที่มี × 120%)
  (คำนวณเฉพาะความต้องการที่พุ่งเกิน 120% เพื่อเติมรถเฉพาะจุดที่ขาดแคลนวิกฤต)

เหตุผลของ 20% Safety Buffer (6 ปัจจัยสำคัญ):
1. ป้องกันเขตต้นทางขาดรถเอง: เก็บรถไว้อย่างน้อย 80% ของ Supply ไม่ดึงออกจนหมด
2. ตัดปริมาณรถที่เข้าช่วงพักงาน (Shift Change / Break): สมมติ ~20% ของ Dropoff จอดพักไม่รับงานต่อ
3. รถที่เดินทางออกจากโซนทันที: แท็กซี่ส่งผู้โดยสารเสร็จแล้วขับออกจากโซนโดยไม่รอรับงาน
4. รองรับ Demand Spikes ภายในชั่วโมง: ป้องกันความผันผวนของความต้องการในแต่ละนาที
5. ชดเชยความคลาดเคลื่อนของ Proxy Measurement: Dropoff เป็นเพียงตัวแทน ไม่ใช่สถานะรถว่างจริง
6. สมดุลเชิงโครงสร้างระหว่าง Excess และ Shortfall thresholds

3.3 ตารางตัวเลขควบคุมทางกายภาพ (Magic Numbers Matrix & Rationale)

เพื่อให้แผนจัดสรรแท็กซี่นำไปปฏิบัติได้จริง ระบบกำหนดเงื่อนไขทางกายภาพและเวลาที่สอดคล้องกันอย่างสมบูรณ์ ดังตารางต่อไปนี้:

ค่าพารามิเตอร์ (Magic Number)	ค่าที่กำหนดในระบบ	การแปลงหน่วย/คำนวณ	เหตุผลรองรับเชิงวิศวกรรมและกายภาพ
Max Distance Threshold	0.15 degrees	0.15° × 111 km/° = 16.6 km (Euclidean) 16.6 km × 1.3 (Road Factor) ≈ 21.6 km (Road)	ครอบคลุมคู่โซนติดกัน (เช่น Manhattan ↔ Brooklyn 12km) แต่บล็อกโซนไกลเกินจริง (Staten Island 30+km)
Average City Velocity	40 km/h	ความเร็วเฉลี่ยรวมถนนและทางด่วน (Rush Hour 18-28 km/h, ทางด่วน 65 km/h)	จุดสมดุลที่ไม่ต่ำเกินไป (ถ้าใช้ 28km/h จะกรองมากเกินเหลือแค่ 14km) และไม่สูงเกินไป (ถ้าใช้ 50km/h จะย้ายไกลเกินจริง)
Travel Time Cap	30 นาที	Est. Travel Time = (Dist_km / 40) × 60 ถ้า time > 30 min → Skip Route	คิดเป็น 50% ของช่วง Rush Hour (2-3 ชม.) ถ้าย้ายใช้เวลาเกิน 30 นาที จะเสียโอกาสรับผู้โดยสารและไปถึงหลัง Peakจบ
Minimum Allocation	> 10 ทริป	Amount > 10 trips / route	ป้องกันการย้ายทริปรถจำนวนน้อยเกินไป ซึ่งไม่คุ้มค่ากับการสื่อสารและการประสานงาน (Management Overhead)
Safety Buffer	20% (0.8 / 1.2)	Excess = (Supply × 0.8) - Demand Shortfall = Demand - (Supply × 1.2)	สำรองรถ 80% ไว้ที่เขตเดิมเพื่อป้องกันเขตต้นทางกลายเป็นโซนขาดแคลนเอง
Proactive Departure Buffer	5 นาที	T_depart = T_target - (T_travel + 5 min)	สำรองเวลากันความล่าช้าจากการจราจร เพื่อให้รถแท็กซี่ไปถึงปลายทางก่อนเริ่มช่วง Peak Hour จริง

สมการพิสูจน์ความสอดคล้องทางกายภาพ (Consistency Proof Check):

การกำหนดค่าพารามิเตอร์ทั้ง 3 ตัวข้างต้นมีความสอดคล้องกันอย่างสมบูรณ์ตามสมการ:

Travel Time = (0.15° × 111 km/° × 1.3) / 40 km/h × 60 นาที = 32.37 นาที ≈ 30 นาที

พิสูจน์ว่าข้อกำหนดระยะทาง 0.15°, ความเร็ว 40 km/h และเวลาเดินทาง 30 นาที มีความสอดคล้องทางคณิตศาสตร์และกายภาพอย่างแท้จริง

3.4 สถาปัตยกรรมทางภูมิศาสตร์ และ Borough Adjacency Matrix

เพื่อป้องกันไม่ให้ระบบแนะนำให้รถแท็กซี่ขับข้ามแม่น้ำ Hudson หรือข้ามพื้นที่ที่ไม่มีสะพานเชื่อมต่อโดยตรง ระบบได้สถาปนา Borough Adjacency Matrix เพื่อควบคุมขอบเขตการย้ายรถระหว่างเขตดังนี้:

เขตต้นทาง (Source Borough)	เขตปลายทางที่อนุญาตให้ย้ายรถไปได้ (Allowed Adjacent Boroughs)	เงื่อนไขพิเศษและการบล็อก
Manhattan	Manhattan, Bronx, Brooklyn, Queens	อนุญาตย้ายไปยัง 4 เขตหลักที่มีสะพาน/อุโมงค์เชื่อมต่อ
Brooklyn	Brooklyn, Manhattan, Queens	อนุญาตย้ายในเขตข้างเคียง บล็อกการย้ายไป Bronx โดยตรง (ไกลเกิน 0.15°)
Queens	Queens, Manhattan, Brooklyn	อนุญาตย้ายไปยัง Manhattan และ Brooklyn บล็อกข้ามไป Bronx
Bronx	Bronx, Manhattan	อนุญาตย้ายเข้า Manhattan บล็อกการย้ายข้ามไป Queens/Brooklyn โดยตรง
Staten Island	Staten Island เท่านั้น	❌ บล็อกการย้ายออก 100% (ไม่มีสะพานเชื่อมทางตรง ต้องผ่านตัวกลาง)
EWR (Newark)	EWR เท่านั้น	❌ บล็อกการย้ายออก 100% (ข้ามรัฐ New Jersey ข้ามแม่น้ำ Hudson)

3.5 Wave Deployment System (ระบบปล่อยรถย่อย 6 ช่วง × 10 นาที)

แทนที่จะปล่อยแท็กซี่ทั้งหมดพร้อมกันในช่วงเริ่มชั่วโมง ซึ่งจะทำให้เกิดการจราจรคอขวด (Bottleneck) และความแออัดบริเวณจุดจอดรอ (Staging Area) ระบบได้ออกแบบ Front-Loaded Wave Deployment Policy โดยแบ่งการปล่อยรถออกเป็น 6 ช่วงย่อย ช่วงละ 10 นาที สอดคล้องกับพฤติกรรมผู้โดยสารที่ต้องการรถมากที่สุดในช่วงต้นชั่วโมงเลิกงาน:

Wave	ช่วงเวลา (นาที)	สัดส่วนน้ำหนัก (%)	ตรรกะพฤติกรรมผู้โดยสาร (Passenger Dynamics)
Wave 1	นาทีที่ 00–10	25%	กลุ่ม Peak Burst เลิกงานตรงเวลา ต้องการรถเร็วที่สุด คิวยาวสุด
Wave 2	นาทีที่ 10–20	25%	กลุ่มเก็บของ/ลงลิฟต์ตามลงมา Demand ยังแน่นสูง
Wave 3	นาทีที่ 20–30	20%	กลุ่ม Demand ปานกลาง เคลียร์คิวสะสม
Wave 4	นาทีที่ 30–40	15%	กลุ่ม Demand เริ่มชะลอตัว
Wave 5	นาทีที่ 40–50	10%	กลุ่มเดินทางทั่วไป / ไม่รีบเร่ง
Wave 6	นาทีที่ 50–60	5%	กลุ่มตกค้างท้ายชั่วโมง เตรียมเข้าสู่รอบชั่วโมงใหม่

สูตรคำนวณเวลาออกเดินทางของแต่ละ Wave (Wave Departure Calculation):

Departure_Min = (Target_Hour × 60) + Arrival_Start_Min - Est_Travel_Min - 5 (Buffer)

ตัวอย่าง: ย้ายแท็กซี่ 150 ทริป ไป Penn Station ในเป้าหมาย 18:00 น. (ใช้เวลาเดินทาง 15 นาที) → Wave 1 (38 ทริป) ออกเดินทาง 17:40 น. ถึง 18:00 น. / Wave 2 (38 ทริป) ออกเดินทาง 17:50 น. ถึง 18:10 น.

3.5 ข้อพิจารณาทางสถาปัตยกรรม: เหตุผลที่ไม่เลือกใช้ข้อมูลระดับความละเอียดรายนาที (Justification for Temporal Granularity & Big O Trade-off Analysis)

ในการออกแบบสถาปัตยกรรมระบบจัดสรรแท็กซี่ **NYC Yellow Taxi Demand-Supply Framework** การเลือกสเกลความละเอียดด้านเวลา (Temporal Granularity) เป็นการตัดสินใจเชิงวิศวกรรมที่ต้องถ่วงน้ำหนักระหว่าง **ความซับซ้อนในการประมวลผล (Computational Complexity)** และ **ความแม่นยำเชิงปฏิบัติการ (Operational Feasibility)** ระบบจึงเลือกใช้ **Hourly Aggregation (รายชั่วโมง)** ควบคู่กับ **10-Minute Wave Deployment** แทนการประมวลผลระดับรายนาที (Minute-Level Granularity) โดยมีเหตุผลรองรับทางเทคนิคดังนี้:

---

1. การวิเคราะห์ความซับซ้อนเชิงอัลกอริทึม (Big O & Computational Complexity Analysis)

- **อัลกอริทึมจับคู่ Spatial-Temporal Matching ($O(Z^2 \times T)$):** อัลกอริทึม Greedy Distance-Based Matching ทำการคำนวณจับคู่ระหว่างโซนล้นเกิน ($Z_{Overflow}$) และโซนขาดแคลน ($Z_{Shortage}$) บนโซนย่อยทั้งหมด $Z = 263$ โซน ในทุกหน้าต่างเวลา T:
    - $TimeComplexity = O(Z_{Overflow} \times Z_{Shortage} \times T) \approx O(Z^2 \times T)$
    - **หากใช้ Hourly Aggregation ($T = 24$ ช่วงเวลา/วัน):**

      $$Operations/Day = 263 \times 263 \times 24 \approx 1.66 \times 10^6 \text{การคำนวณ/วัน}$$

      (ใช้เวลาประมวลผลบน PySpark/Numpy Vectorization เพียง < 0.1 วินาที)

    - **หากขยายเป็น Minute-Level Aggregation ($T = 1,440$ ช่วงเวลา/วัน):**

      $$Operations/Day = 263 \times 263 \times 1,440 \approx 9.96 \times 10^7 \text{การคำนวณ/วัน}$$

      *(ความซับซ้อนเชิงเวลาพุ่งสูงขึ้นถึง **60 เท่า** โดยไม่สร้างคุณค่าทางการจัดสรรเพิ่มขึ้น)*

- **วิกฤต $O(N^2)$ Data Shuffle บน Big Data Cluster:** หากระบบประมวลผลระดับรายนาทีบนชุดข้อมูลทริปการเดินทางดั้งเดิม N (~8.7 ล้านรายการ) การทำ Spatial Cartesian Join หรือ Cross-Join ระหว่างตำแหน่ง Pickup และ Dropoff แบบไมโครวินาที จะเกิดความซับซ้อนระดับ $O(N^2)$ ซึ่งสร้างภาระ **Data Shuffle Spill ข้าม Worker Nodes บน Databricks** มหาศาล ส่งผลให้ Memory OOM (Out-of-Memory) และบั่นทอนประสิทธิภาพการแสดงผลแบบ Real-time บน Streamlit Dashboard

---

2. สถาปัตยกรรมข้อมูลและข้อจำกัดด้าน I/O (Data Pipeline & Partitioning Overhead)

- **ปัญหาข้อมูลเบาบาง (Data Sparsity & Zero-Inflation):** การย่อยข้อมูลลงระดับ 1 นาทีใน 263 โซนย่อย จะทำให้เมทริกซ์ข้อมูลส่วนใหญ่กลายเป็นค่าว่าง (Zero-inflated Sparse Matrix) เนื่องจากโซนส่วนใหญ่ไม่ได้มีทริปเกิดขึ้นทุกๆ นาที
- **ปัญหา Small File Problem ใน Delta Lake:** การจัดกลุ่ม (GroupBy) และพาร์ติชันข้อมูลระดับรายนาทีจะทำให้เกิดไฟล์ Parquet ขนาดเล็กจำนวนหลายล้านไฟล์ (Small Files Problem) ส่งผลให้ Spark Vectorization Engine และ I/O Read Throughput ช้าลงอย่างเห็นได้ชัด

---

3. อัตราส่วนสัญญาณต่อสัญญาณรบกวนทางสถิติ (Signal-to-Noise Ratio - SNR)

- **High-Frequency Random Noise:** ข้อมูลระดับรายนาทีเต็มไปด้วย **Random Noise** จากปัจจัยชั่วคราว เช่น สัญญาณไฟจราจรติดขัด 60 วินาที, การชะลอจอดรอผู้โดยสาร หรือความคลาดเคลื่อนของเวลาบนมิเตอร์แท็กซี่ การใช้ข้อมูลรายนาทีจะทำให้โมเดลเกิดภาวะ **Overfitting** กับสภาวะสุ่มชั่วคราว
- **Statistical Smoothing:** การยุบรวมข้อมูลเป็นรายชั่วโมง (Hourly Aggregation) ช่วยทำการ **Signal Smoothing** หักล้าง Noise สุ่มเหล่านั้นออก ตามกฎสถิติ Law of Large Numbers เหลือไว้เฉพาะ **Macro Demand Pattern** ที่แท้จริงและเสถียรพอสำหรับการพยากรณ์ด้วย SARIMA

---

4. ความสอดคล้องทางกายภาพและการปฏิบัติการจริง (Operational & Staging Feasibility)

- **พฤติกรรมมนุษย์และการวนรถ (Driver Turnover Dynamics):** ในโลกจริง แท็กซี่และผู้โดยสารไม่สามารถตอบสนองต่อคำสั่งระดับรายนาทีได้ การเทียบจอด ชำระเงิน และถ่ายถ่ายผู้โดยสารขึ้น-ลงรถ ใช้เวลาเฉลี่ย 2–5 นาทีต่อทริป
- **จุดสมดุลด้วย 10-Minute Wave Deployment:** ระบบจึงแก้ปัญหาระดับปฏิบัติการย่อยด้วยการใช้ **Wave Deployment Strategy (แบ่ง 1 ชั่วโมงเป็น 6 ช่วง × 10 นาที)** ซึ่งเป็นการดึงข้อดีของทั้งสองฝั่ง:
    1. ใช้ **Hourly Data** ในการพยากรณ์โครงสร้างภาพรวม (Macro Forecasting)
    2. ใช้ **10-Minute Interval** ในการทยอยปล่อยรถเป็นล็อตย่อยเพื่อป้องกันการจราจรคอขวด (Micro Operations)

4. Economics & Mechanism Design (โครงสร้างเศรษฐศาสตร์และกลไกจูงใจ)

หน่วยนับ: Trips vs คันรถ

- ระบบนับเป็น **trips** (dropoff count จากข้อมูล) ไม่ใช่จำนวนแท็กซี่จริง
- แท็กซี่ 1 คันทำเฉลี่ย ~2 trips/ชม. ในช่วงเร่งด่วน (ข้อมูล TLC)
- ตัวอย่าง: 25,675 trips ≈ 12,800 คัน (ใกล้เคียง NYC fleet จริง ~13,500 medallions)

4.1 โครงสร้างต้นทุนและกลไกจูงใจ (Unit Economics & Driver Cost Structure)

การประเมินความคุ้มค่าของการย้ายรถแท็กซี่ ไม่ได้มองเพียงแค่ปริมาณรถ แต่คำนวณจากต้นทุนการขับรถว่าง (Cruising Cost) รายทริป ดังนี้:

1. ค่าน้ำมันและการสึกหรอ (Fuel & Depreciation Cost): กำหนดไว้ที่ $0.15 / กิโลเมตร ($0.065 ค่าน้ำมัน + $0.085 ค่าเสื่อมสภาพ)
2. ค่าเสียโอกาสของคนขับ (Driver Opportunity Cost): กำหนดไว้ที่ $30.00 / ชั่วโมง คำนวณตามระยะเวลาเดินทางว่าง (Est. Travel Time / 60) * $30
3. สูตรต้นทุนการย้ายรถรวมต่อเส้นทาง: Allocation_Cost = Amount * [((Travel_Time_Min / 60) * $30) + (Distance_km * $0.15)]
4. Driver Net Earning Ratio (65%): คนขับได้รายได้สุทธิ 65% หลังหักค่าน้ำมัน (~20%), ค่าเสื่อมราคา (~10-15%) และค่าธรรมเนียม (~5%) โดย Standard Trip ได้ $13 × 65% = $8.45 และ JFK Flat Trip ได้ $52 × 65% = $33.80

4.2 สูตรคำนวณรายได้ดึงกลับ (Revenue Recovered Logic)

เมื่อแท็กซี่เดินทางไปถึงโซนปลายทางที่ขาดแคลน รายได้ที่จะดึงกลับมาได้ในชั่วโมงนั้น คำนวณจาก:
• เวลาให้บริการคงเหลือในชั่วโมงนั้น: Avail_Min = max(0, 60 - Travel_Time_Min)
• จำนวน เที่ยววิ่งที่ทำได้ต่อทริป: Trips_per_Taxi = max(1, Avail_Min / 15 นาที) [กำหนดเวลาเฉลี่ย 15 นาที /trip]
• อัตราค่าโดยสารเฉลี่ย (Avg Fare): $52.00 สำหรับโซนสนามบิน JFK (Flat Rate) และ $13.00 สำหรับ Standard Local Trip ที่กำหนดโดย TLC
• สูตรรายได้ที่กู้คืนรวม: Revenue_Recovered = Amount * Trips_per_Taxi * (Avg_Fare + Destination_Avg_Tip)

4.3 การประเมินกำไรสุทธิต่อทริป (Net Benefit per Trip Evaluation)

ระบบประเมินเกณฑ์ความคุ้มค่าจาก Net Benefit per Trip = (Revenue_Recovered - Allocation_Cost) / Amount โดยแบ่งเป็น 3 ระดับ:
• ✅ คุ้มค่าสูง (High Benefit): Net Benefit per Trip > $20.00 / ทริป / ชม. (แท็กซี่ทำรายได้คืนมา 2–3 เท่าของค่าย้าย)
• 🟡 คุ้มค่าปานกลาง (Moderate): $0.00 < Net Benefit per Trip <= $20.00 / ทริป
• ❌ ไม่คุ้มค่า (Negative): Net Benefit per Trip <= $0.00 / ทริป (เสียเงิน ค่าย้ายสูงกว่ารายได้)

4.4 Refined Framing: Opt-In Demand Guidance Mechanism (การชี้แหล่งรับผู้โดยสารโดยตรง)

Core Value Proposition Refinement

ปรับปรุงกรอบแนวคิดหลัก (Refined Value Proposition):

1. Direct Demand Matching & Cost/Time Saving (หัวใจหลัก): ระบบไม่ใช่การสั่งการบังคับ (Command-based) และไม่ได้สร้างขึ้นเพื่อไล่ล่าทิปสนามบินเป็นหลัก แต่เป็น Opt-In Demand Guidance System& ที่ชี้เป้าหมายโซนที่มีผู้โดยสารรอรถอยู่จริงให้คนขับเดินทางไปรับงานได้ทันทีโดยไม่ต้องขับรถวนหาผู้โดยสารอย่างเสี่ยงดวง (Cruising Rate 30-40%) ซึ่งช่วยประหยัดค่าน้ำมัน $0.15/km$)และลดค่าเสียโอกาสเวลา(30/hr) ได้โดยตรง

2. Tip & Airport Fare (แรงจูงใจเสริม): ค่าทิป (+24.4% ใน Queens) และค่าโดยสารอัตราพิเศษสนามบิน ($52 Flat Rate) ถือเป็นส่วนต่างกำไรเสริม (Supplementary Margin) ที่ถูกบวกเข้าไปในสมการ Revenue per Trip อยู่แล้ว ไม่ใช่เหตุผลเดี่ยวในการย้ายรถ ทำให้กลไกตลาด (Market-Based Mechanism) มีความสมเหตุสมผลสำหรับคนขับทุกคน

5. System Architecture & Interactive Simulation Framework (สถาปัตยกรรมระบบและการจำลองสถานการณ์)

5.1 System Delivery & Simulation Results

ระบบถูกสร้างและปรับใช้งานจริงบน Databricks ในรูปแบบ Streamlit Web App (nyc-taxi-what-if-dashboard) และ Lakeview Dashboard โดยในแอปพลิเคชันมี 4 Summary Cards หลัก:

cb1: รายได้หลังมีระบบ (USD/ชม.) รายได้รวมของแท็กซี่ที่ย้ายไปโซนใหม่	**cb2: รายได้ที่กู้คืนได้ (USD/ชม.)** กำไรเพิ่มเติมสุทธิจากการมีระบบจัดสรร
cb3: Net/Trip (USD/ทริป/ชม.) กำไรสุทธิต่อ 1 ทริป (มี Label คุ้มค่า/พอใช้)	**cb4: รายได้เดิมก่อนมีระบบ (USD/ชม.)** รายได้เดิมที่แท็กซี่ได้หากไม่มีการจัดสรรย้ายโซน

สรุปผลการจำลองสภาวะ Evening Rush Hour Case Study (17:00–19:00 น.):

จากการจัดสรรแท็กซี่ 25,675 ทริป ผ่าน 169 เส้นทาง สามารถดึง Demand-Supply Ratio ของเขต Queens จากสภาวะวิกฤต **1.71 ลงมาเหลือ 1.00 (สภาวะสมดุลสมบูรณ์)** และช่วยดึงเขตล้นเกิน (Brooklyn/Bronx) ให้กลับเข้าสู่สภาวะสมดุล รวมปรับปรุงได้ถึง 12 เขต/ช่วงเวลา สร้างผลประโยชน์สุทธิทางการเงินรวม (Net Benefit) สูงถึง **~$2,163,489 (ประมาณ 78 ล้านบาท)** ในช่วง Evening Rush เดียว

5.2 อัลกอริทึมการจัดสรรแบบ Greedy Distance-Based Matching

การจัดสรรรถแท็กซี่ใช้หลักการ Greedy Matching โดยเรียงลำดับความสำคัญตามระยะทาง (ส่งรถจากโซนล้นเกินไปยังโซนขาดแคลนที่ใกล้ที่สุดก่อน) ผ่านลำดับการกรอง 4 ชั้น (4-Layer Filters):

Step 1: คัดแยก Overflow Zones (Ratio < 0.8) และ Shortage Zones (Ratio > 1.2)
Step 2: คำนวณ Excess (พร้อม Buffer 20%) และ Shortfall
Step 3: สร้าง Distance Matrix ด้วย Numpy Vectorization (คำนวณรวดเร็วโดยไม่ใช้ Loop)
Step 4: ใช้ Filter 4 ชั้น: (1) Adjacency Check -> (2) Distance <= 0.15° -> (3) Travel Time <= 30 min -> (4) Amount > 10 ทริป
Step 5: คำนวณ Cost-Benefit, Departure Time และอัปเดตสถานะ Ratio ใหม่หลังการจัดสรร

5.3 ผลการจำลองสถานการณ์ Before vs After (Evening Rush Case Study)

จากการจำลองการจัดสรรในช่วงเวลาเร่งด่วนเย็น (Evening Rush 17:00–19:00 น.) ซึ่งเป็นช่วงวิกฤตที่สุด ผลลัพธ์ก่อนและหลังการจัดสรรสรุปได้ดังตาราง:

เขตพื้นที่ (Borough)	Ratio ก่อนจัดสรร (Before)	Ratio หลังจัดสรร (After)	การเปลี่ยนแปลง	สถานะหลังจัดสรร	ผลกระทบเชิงระบบ
Queens	1.71 (ขาดแคลนหนัก)	1.00 (สมดุล)	-0.71	🟢 สมดุล (Balanced)	ได้รับรถเติมเต็ม 37,027 ทริป ดึง Ratio เข้าสู่สภาวะสมดุลสมบูรณ์
Brooklyn	0.25 (ล้นเกินหนัก)	1.00 (สมดุล)	+0.75	🟢 สมดุล (Balanced)	ระบายรถส่วนเกิน 37,027 ทริป ไปช่วย Queens ลดการขับวนเปล่า
Bronx	0.13 (ล้นเกินหนัก)	0.67 (ดีขึ้น)	+0.54	🔵 ล้นเกินลดลง	ระบายรถส่วนเกิน 5,001 ทริป ไปช่วย Manhattan
Manhattan	1.01 (สมดุล)	1.00 (สมดุล)	-0.01	🟢 สมดุล (Balanced)	คงสภาวะสมดุล ได้รับรถเสริมเล็กน้อยจาก Bronx/Brooklyn
EWR / Staten Island	0.02 / 0.11	0.02 / 0.11	0.00	🔵 คงเดิม	ไม่มีการย้ายออกตามเงื่อนไข Adjacency Matrix

EVENING RUSH SIMULATION FINANCIAL IMPACT

สรุปผลการจำลองสภาวะรวมทั้งเมืองในช่วง Evening Rush:

- จำนวนเส้นทางย้ายรถที่ผ่านเงื่อนไข: 169 เส้นทาง
- ปริมาณรถแท็กซี่ที่ได้รับการจัดสรรรวม: 25,675 ทริป
- รายได้หลังมีระบบ (USD/ชม.) : $1,174,760
- รายได้เดิมก่อนมีระบบ (USD/ชม.) : $1,050,753 (~77.8 ล้านบาท) ในช่วง 2 ชั่วโมงเร่งด่วน
- รายได้ที่กู้คืนได้ (USD/ชม.) : $124,007

6. Time Series Forecasting Models (โมเดลการพยากรณ์อนุกรมเวลา)

6.1 การเปรียบเทียบโมเดล SARIMA vs Prophet (Machine Learning Base)

เพื่อทำการพยากรณ์ Demand ล่วงหน้าสำหรับแผนจัดสรรแบบ Proactive ระบบได้ทดสอบพยากรณ์ข้อมูลอนุกรมเวลารายชั่วโมงด้วย 2 โมเดลหลัก:

โมเดล (Model Architecture)	การตั้งค่าพารามิเตอร์ (Configuration)	ค่า MAPE Score	จุดเด่นและข้อจำกัด
SARIMA	SARIMA(1,1,1)(1,1,1,24) Daily Seasonality (s=24)	~33.0% (ดีปานกลาง)	จับ Pattern รายชั่วโมงและ Rush Hour ได้ดีกว่า แต่เซนซิทีฟต่อข้อมูลสั้น
Prophet	Prophet with Daily + Weekly Seasonality	~39.5% (ด้อยกว่า)	รองรับ Trend เปลี่ยนแปลงได้ดี แต่ให้ Error สูงกว่าเมื่อข้อมูล Train มีเพียง 25 วัน

6.2 วิเคราะห์สาเหตุที่ค่า MAPE สูง (High Variance Root Cause Analysis)

1. ข้อมูล Training Data มีจำกัดเพียง 25 วัน (600 ชั่วโมง) ทำให้โมเดลเรียนรู้ Pattern ระยะยาวไม่เพียงพอ
2. ข้อมูลอนุกรมเวลารายชั่วโมง (Hourly Trips) มี Variance และ Noise สูงมากตามธรรมชาติ
3. ยังไม่ได้รวมปัจจัยภายนอก (Exogenous Variables) เช่น สภาพอากาศ (Weather), วันหยุดเทศกาล (Holidays) และตารางเที่ยวบิน
4. SARIMA ยังไม่ได้แยก Pattern ระหว่าง วันธรรมดา (Weekday) และวันหยุด (Weekend) ส่งผลให้ประเมิน Evening Peak ต่ำกว่าความเป็นจริงเล็กน้อย

6.3 ทำไมจึงเลือกใช้สถิติโมเดล (SARIMA) แทน Machine Learning ทั่วไป

ในการเลือกระเบียบวิธีสำหรับการพยากรณ์ Demand ของระบบ ทีมวิจัยได้ทำการเปรียบเทียบเชิงลึกระหว่างการใช้โมเดลทางสถิติ (Statistical Time Series Models) อย่าง SARIMA กับโมเดล Machine Learning / Deep Learning ทั่วไป (เช่น XGBoost, Prophet, LSTM) โดยมีเหตุผลสนับสนุนและผลวิเคราะห์เปรียบเทียบดังนี้:

1. ข้อจำกัดด้านขนาดข้อมูล (Data Volume Constraint): ชุดข้อมูล Training Data ในระบบมีระยะเวลาเพียง 25 วัน (600 ชั่วโมง) และเมื่อรวมเต็มเดือนมกราคมจะมีเพียง 744 ชั่วโมง ซึ่งโมเดลประเภท Deep Learning หรือ XGBoost จำเป็นต้องใช้ข้อมูล Time Series ย้อนหลังอย่างน้อยหลายฤดูกาล (หลายพันถึงหมื่นชั่วโมง) เพื่อเรียนรู้ Pattern ที่ซับซ้อนโดยไม่เกิดปัญหา Overfitting การใช้ SARIMA จึงเหมาะสมที่สุดสำหรับ Data Footprint ขนาดสั้นถึงปานกลาง

2. ความสามารถในการอธิบายผลลัพธ์ (Interpretability & White-Box Nature): SARIMA เป็นสถิติโมเดลที่มีโครงสร้างทางคณิตศาสตร์ชัดเจน สามารถแยกแยะองค์ประกอบด้าน Auto-Regression (p, P), Differencing (d, D) เพื่อขจัด Non-stationarity และ Moving Average (q, Q) ได้อย่างโปร่งใส ต่างจากโมเดล Deep Learning ซึ่งเป็น Black-Box ที่ยากต่อการอธิบายกลไกภายในให้คณะกรรมการประเมินหรือผู้จัดทำนโยบายสาธารณะเข้าใจ

3. ผลการวัดประสิทธิภาพเชิงเปรียบเทียบ (MAPE Benchmark): จากการทดสอบเปรียบเทียบกับ Prophet บนชุดข้อมูลจริง SARIMA สามารถทำค่า Mean Absolute Percentage Error (MAPE) ได้ที่ 32.57% ซึ่งมีประสิทธิภาพเหนือกว่า Prophet ที่ได้ MAPE สูงถึง 39.54%ถจับ Seasonality รายวัน (s=24) และความสัมพันธ์ของ Hourly Peak ได้แม่นยำกว่า เนื่องจาก SARIMA สามารถ

- เหตุใดจึงไม่ใช้ Deep Learning / XGBoost: ข้อมูล 744 ชั่วโมงไม่เพียงพอสำหรับการ Train Deep Learning ให้เกิด Convergence โดยไม่ Overfit และการปรับ Hyperparameter ของ XGBoost บน Time Series สั้นเสี่ยงต่อการหลุด Pattern รายชั่วโมง
- เหตุใด MAPE ~33% จึงเป็นค่าที่ยอมรับได้: ข้อมูล Taxi Demand รายชั่วโมงมีความผันผวน (High Variance/Noise) สูงมากตามธรรมชาติ และระบบ Allocation นำพยากรณ์ไปใช้ร่วมกับ Safety Buffer 20% ร่วมกับ Macro Pattern ไม่ได้พึ่งพาตัวเลขโดดๆ ทำให้ระดับ Error ที่ 33% เพียงพอในการชี้นำสภาวะ Over/Under Demand ได้อย่างมีประสิทธิภาพ

7. Positive Externalities (คุณค่าทางอ้อมเชิงสังคมและสิ่งแวดล้อม)

นอกเหนือจากผลประโยชน์ตรงทางการเงิน ($2.16M Net Benefit) การจัดสรรด้วยระบบ Proactive Wave Deployment ยังสร้าง **ผลดีทางอ้อม (Positive Externalities)** ในมิติเศรษฐศาสตร์เมือง (Urban Economics) และนโยบายสาธารณะ 4 ด้านสำคัญ:

ตารางที่ 7.1: ผลกระทบทางอ้อมเชิงคุณภาพและนโยบายสาธารณะ (Qualitative Externalities Matrix)

มิติผลกระทบ (Dimension)	กลไกการเกิดผลกระทบ (Mechanism)	ผลลัพธ์ทางอ้อมเชิงคุณภาพ (Qualitative Outcome)
🍃 สิ่งแวดล้อม & มลพิษ (Environmental Impact)	ลดระยะการขับวนหารถเปล่า (Cruising Rate 30-40%) ในโซนล้นเกิน (Brooklyn/Bronx)	ลดการปล่อยก๊าซเรือนกระจก (CO_2), ฝุ่น PM2.5 และการสิ้นเปลืองพลังงานน้ำมันเปล่าประโยชน์
🚗 การจราจรในเมือง (Traffic Congestion)	ลดจำนวนแท็กซี่ว่างที่ขับชะลอช้าๆ บนถนนสายหลักเพื่อสแกนหาผู้โดยสาร	ช่วยให้การไหลเวียนของการจราจร (Traffic Flow) ในเขตเมืองและคอขวดสะพานดีขึ้น
⏱️ คุณภาพชีวิตผู้โดยสาร (Passenger Experience)	เติม Supply ล่วงหน้าก่อน Rush Hour ในจุดวิกฤต (JFK/LGA/Penn Station)	ลดเวลารอคอย (Wait Time) และลดความเครียดจากการตกรถ/ตกเครื่องของผู้โดยสาร
💰 สวัสดิภาพคนขับ (Driver Welfare Efficiency)	แนะนำเส้นทางที่มี Demand สูงและ Tip สูง (+24.4% ใน Queens) ผ่าน Tip Map	ยกระดับรายได้สุทธิต่อชั่วโมงของคนขับ โดยไม่ต้องขยายชั่วโมงการทำงานให้เหน็ดเหนื่อย

ประเด็นเหล่านี้เป็น 'Qualitative Externalities' เนื่องจากชุดข้อมูล TLC ไม่ได้มีตัววัดก๊าซเรือนกระจกรายคันโดยตรง

8. System Limitations

8.1 ตารางสรุปข้อจำกัดทางเทคนิคและแนวทางแก้ไข (System Limitations Matrix)

ข้อจำกัดทางเทคนิค (Limitation)	ระดับความสำคัญ	สาเหตุและบริบทข้อมูล	แนวทางแก้ไขในอนาคต (Future Resolution)
Supply ใช้ Dropoff Count เป็น Proxy	สูง (High)	ไม่มีข้อมูลสถานะรถว่างจริง (Cruising) หรือ GPS Ping จากผู้ขับขี่	เชื่อมต่อข้อมูล Live GPS Pings หรือข้อมูล High-Volume FHV (Uber/Lyft)
ข้อมูลมีเฉพาะเดือนมกราคม	สูง (High)	ข้อจำกัดของชุดข้อมูลที่ได้รับมาทดสอบ	ขยายการดึงข้อมูลครอบคลุม 12 เดือน เพื่อศึกษา Seasonal Pattern รายปี
Simulation เป็น Deterministic Model	สูง (High)	สมมติว่าคนขับปฏิบัติตามแผน 100% ยังไม่ได้จำลองการตัดสินใจคนขับ	พัฒนาเป็น Agent-Based Simulation (ABS) เพื่อจำลองพฤติกรรมมนุษย์
พยากรณ์ยังไม่ได้เชื่อมอัตโนมัติเข้า Streamlit	กลาง (Medium)	แอป Streamlit ยังใช้อัตราส่วนย้อนหลังจาก Parquet	พัฒนา Pipeline ให้พยากรณ์ส่งค่าตรงเป็น Input ของ Allocation Engine
ไม่มี Capacity Constraint ของแท็กซี่รวม	กลาง (Medium)	ไม่ได้จำกัดจำนวนแท็กซี่สูงสุดในเมือง (NYC Medallion Cap ~13,500)	ใส่ขอบเขต Total Taxi Capacity ในสมการจัดสรร
ไม่มีข้อมูลสภาพอากาศ (Weather)	กลาง (Medium)	ไม่มีชุดข้อมูลสภาพอากาศประกอบ	ปรับปรุงโมเดลพยากรณ์เป็น SARIMAX โดยเพิ่มคอลัมน์ฝน/หิมะ

9. Future Roadmap & Productionization (แผนการพัฒนาต่อยอดสู่อุตสาหกรรม)

เพื่อยกระดับโครงงานนี้จากโปรเจกต์ต้นแบบ (Proof of Concept) สู่อยู่ในระดับอุตสาหกรรมจริง (Production-Grade System) ได้วางแผนการพัฒนาต่อยอดออกเป็น 3 มิติหลัก:

1. Real-Time Telemetry Ingestion: เชื่อมต่อข้อมูลตำแหน่งรถแท็กซี่จาก GPS Pings และสถานะไฟว่าง (Meter Status) แบบ Real-time ผ่าน Apache Kafka และ Spark Streaming เพื่อเปลี่ยน Supply จาก Proxy เป็น Actual Vacant Taxis

2. Agent-Based Simulation (ABS): พัฒนาแบบจำลองการตัดสินใจของคนขับแท็กซี่รายบุคคล (Driver Agents) โดยใส่ปัจจัยราคาน้ำมัน ค่าทิป และความพึงพอใจ เพื่อทดสอบว่าคนขับจะตอบสนองต่อ Tip Map และ Reallocation Recommendation อย่างไร

3. Exogenous Variable Integration & Deep Learning: ปรับปรุงโมเดลพยากรณ์โดยรวมข้อมูลสภาพอากาศ (Rain/Snow), ตารางเที่ยวบิน Real-time ของสนามบิน JFK/LGA, วันพิเศษต่างๆและตารางจัดแสดงคอนเสิร์ต/กีฬา พร้อมทดสอบโมเดลกลุ่ม Deep Learning เช่น Temporal Fusion Transformer (TFT) หรือ LSTM

แนวทางการตอบคำถามสัมภาษณ์ / กรรมการ (20 Technical Q&A Defense Guidelines)

Q1: Supply ใช้ dropoff count เป็น proxy — แปลว่าอะไรและมีปัญหาไหม?

A1: Dropoff count หมายถึงรถที่พึ่งมาส่งผู้โดยสารในโซนนั้น โดยสมมติว่าพร้อมรับงานต่อ ข้อจำกัดคือรถอาจขับกลับทันทีหรือพักงาน แต่ในฐานะ Proxy ถือเป็นตัวแทนที่มีความสัมพันธ์สูงกับจุดที่มีรถปลดไฟว่าง

Q2: ข้อมูลแค่เดือนมกราคมเพียงพอไหม?

A2: เป็นชุดข้อมูลเริ่มต้นในการพิสูจน์แนวคิด (POC) แม้ไม่เห็น Seasonal รายปี แต่เพียงพออย่างยิ่งในการสกัด Pattern รายวันและช่วง Rush Hour เพื่อสร้าง Allocation Model

Q3: ทำไมเลือกใช้ ช่วงสมดุล 0.8 – 1.2?

A3: 1.0 คือ Demand เท่ากับ Supply พอดี การยอมรับ Buffer +-20% ช่วยป้องกันการมองโซนที่ Ratio 1.01 ว่าขาดแคลน ซึ่งไม่สมเหตุสมผลในทางปฏิบัติ

Q4: แผนจัดสรรย้ายรถข้ามเขต คนขับจะยอมทำตามจริงหรือ?

A4: ระบบไม่ได้ใช้คำสั่งบังคับ (Command-based) โดยมีทิปเป็นเพียงส่วนต่างกำไรเสริม (Supplementary Margin)หัวใจหลักคือการชี้เป้าหมายจุดรับผู้โดยสารโดยตรงเพื่อลดเวลาขับวน (Cruising)และประหยัดค่าน้ำมัน/ค่าเสื่อมราคา

Q5: Simulation แบบ Before vs After แค่คำนวณ Ratio ใหม่ใช่ไหม?

A5: เป็น Deterministic Simulation เพื่อแสดงผลลัพธ์ในอุดมคติว่าหากย้ายรถสำเร็จตามแผน สภาวะเมืองจะปรับเข้าสู่สมดุลอย่างไร เฟสต่อไปคือทำ Agent-Based Simulation

Q6: SARIMA MAPE 33% ถือว่าดีไหม ทำไมไม่ใช้โมเดลอื่น?

A6: 33% ถือเป็นระดับปานกลางสำหรับข้อมูล Hourly ที่มีความผันผวนสูงและมี Train set เพียง 25 วัน ได้ทดสอบ Prophet แล้วได้ 39.5% ซึ่งด้อยกว่า ในอนาคตจะพัฒนาเป็น XGBoost/LSTM

Q7: ทำไม Queens ขาดแท็กซี่หนัก รู้ได้อย่างไรว่าเกี่ยวกับสนามบิน?

A7: จากการ drill-down ลงระดับ Zone พบว่า JFK มี Ratio 3.35 และ LGA มี Ratio 2.82 หากแยกโซนสนามบินออก โซนท้องถิ่นใน Queens จะมี Ratio ใกล้เคียง 1.0

Q8: ทำไมจำกัดแผนจัดสรรเฉพาะเส้นทางที่มี Amount > 10 ทริป?

A8: เพื่อป้องกัน Management Overhead การย้ายรถ 1-2 ทริปข้ามเขตไม่คุ้มค่ากับการสื่อสารและการประสานงาน

Q9: ทำไมต้องจำกัด Distance 0.15° และ Travel Time 30 นาที?

A9: 0.15° (≈21.6 km ถนน) ครอบคลุมโซนติดกันแต่บล็อกโซนไกลเกินไป ส่วน 30 นาที คิดเป็นครึ่งหนึ่งของ Rush Hour หากเดินทางนานกว่านั้นจะไม่คุ้มค่าเสียโอกาส

Q10: แอพพลิเคชันนี้ใครเป็นผู้ใช้งานหลัก (Target User)?

A10: มี 2 กลุ่มหลัก คือ (1) ผู้บริหารเมือง/TLC ดูภาพรวมเชิงนโยบาย และ (2) คนขับแท็กซี่ ดูจุดขาดแคลนและอัตราทิปเพื่อวางแผนเดินรถ

Q11: ถ้ามีข้อมูลทั้งปี คุณจะปรับปรุงอะไรในระบบ?

A11: ทำ Seasonal Decomposition แยกแผนจัดสรรตาม 4 ฤดูกาล และเพิ่มการวิเคราะห์วันหยุดเทศกาล (High-demand Holidays)

Q12: หากมีเวลาพัฒนาต่อ จะเพิ่มอะไรเป็นสิ่งแรก?

A12: การเชื่อมต่อ Live Streaming GPS Pings และการรวม Staging Area Capacity Constraint ในระบบจัดสรร

Q13: ถ้าไม่ใช้ Demand-Supply Ratio จะใช้ Metric อะไรแทน?

A13: สามารถใช้ Absolute Shortfall (จำนวนคันที่ขาด) หรือ Passenger Wait Time ได้ แต่ Ratio มีข้อดีคือเป็น Normalized Metric เทียบข้ามเขตที่มีขนาดต่างกันได้ดี

Q14: ทำไมระบบถึงยังไม่ใส่ปัจจัยสภาพอากาศ (Weather Data)?

A14: การใส่สภาพอากาศอย่างถูกต้องจำเป็นต้อง retrain โมเดลเป็น SARIMAX และต้องใช้ข้อมูลหลายเดือน เพื่อเรียนรู้ความสัมพันธ์ จึงวางไว้เป็นแผนพัฒนาในเฟสถัดไป

Q15: ทำไมต้องหัก Opportunity Cost 30% ในการคำนวณรายได้ดึงกลับ?

A15: เพราะแท็กซี่ในโซน Overflow ยังคงหารายได้ได้อยู่บ้าง (~30% ของ Shortage Zone) หากไม่หักออก ตัวเลขรายได้ดึงกลับจะพองเกินจริง 20-30%

Q16: ทำไมไม่ใช้เงินอุดหนุน (Subsidy) จูงใจคนขับ?

A16: เพราะโปรเจกต์นี้เป็น Positive-Sum Game รายได้ที่ดึงกลับเข้ากระเป๋าคนขับ 100% โดยสร้าง Net Benefit > $20/ทริป คนขับได้ประโยชน์ ผู้โดยสารได้รถ จึงไม่จำเป็นต้องใช้เงินรัฐอุดหนุน

Q17: หากไม่มีที่จอดรอรับผู้โดยสาร (Staging Area) ในโซนปลายทางจะทำอย่างไร?

A17: ในระยะสั้นเน้นโซน Hub ใหญ่ที่มี Holding Lot (สนามบิน/สถานีรถไฟ) ในระยะยาวสามารถเพิ่ม Layer 5 Staging Capacity Constraint ร่วมกับ Wave Deployment 10 นาที
เพื่อควบคุมปริมาณรถไม่ให้เกินความจุจุดจอด

Q18: ระบบนี้แตกต่างจากการไล่ล่าทิปอย่างไร?

A18: ทิปเป็นเพียงส่วนต่างกำไรเสริม (Supplementary Margin)
หัวใจหลักคือการชี้เป้าหมายจุดรับผู้โดยสารโดยตรงเพื่อลดเวลาขับวน (Cruising)
และประหยัดค่าน้ำมัน/ค่าเสื่อมราคา

Q19: ทำไมต้องมี Wave Deployment System 6 ช่วง × 10 นาที?

A19: เพื่อลดการจราจรคอขวด (Congestion) บริเวณจุดจอดรอ โดยถ่วงน้ำหนัก Front-Loaded (25%, 25%, 20%, 15%, 10%, 5%) สอดคล้องกับพฤติกรรมคนเลิกงานที่รีบเดินทางช่วงต้นชั่วโมง

Q20: ทำไมต้องมี Wave Deployment System 6 ช่วง × 10 นาที?

A20: เพื่อลดการจราจรคอขวด (Congestion) บริเวณจุดจอดรอ โดยถ่วงน้ำหนัก Front-Loaded (25%, 25%, 20%, 15%, 10%, 5%) สอดคล้องกับพฤติกรรมคนเลิกงานที่รีบเดินทางช่วงต้นชั่วโมง

Q21: ทำไมคำนวณ Cost-Benefit แบบ Per-Trip ไม่ใช่ Per-Route?

A21: เพื่อให้ Label ความคุ้มค่า ($20/ทริป) เป็นอิสระจากจำนวนรถในเส้นทาง
ทำให้สเตคโฮลเดอร์เห็นความคุ้มค่าที่แท้จริง

คำถามด้าน Data pipeline

1. "ข้อมูลที่ใช้มาจากไหน และทำไมใช้แค่เดือนมกราคม?"

ตอบ: NYC TLC Yellow Taxi Trip Records จาก Kaggle มี 4 ปี (2015–2018) แต่ข้อมูลแค่เดือนมกราคม เพราะอาจารย์ให้ข้อมูลแค่นี้

ถามตาม: "ถ้าแค่เดือนมกราคม จะ represent ทั้งปีได้ไหม? ฤดูร้อน/หนาว demand ต่างกันไหม?"

2. "Demand และ Supply คำนวณอย่างไร?"

ตอบ: Demand = count pickup ต่อ zone × hour, Supply = count dropoff ต่อ zone × hour

ถามตาม: "Supply ที่นับจาก dropoff แปลว่าแท็กซี่ที่มาส่งผู้โดยสารแล้วว่างเลยเหรอ? แท็กซี่ที่จอดรออยู่เฉยๆ ไม่ได้ไปส่งใครล่ะ?"

3. "ทำไมเลือก threshold 1.2 และ 0.8?"

ตอบ: 0.8–1.2 = สมดุล, >1.2 = Shortage, <0.8 = Overflow

ถามตาม: "เลขพวกนี้มาจากไหน? มี paper อ้างอิงไหม หรือตั้งเอง?"

4. "Supply เป็น dropoff count แต่ใน cost-benefit คำนวณรายได้เป็น per trip — จำนวนนี้คือรถหรือ trip?"

ตอบที่ควรตอบ: เป็น trips ไม่ใช่คันรถ — แท็กซี่ 1 คันทำ ~2 trips/ชม. ตาม TLC data

ถามตาม: "งั้นถ้า amount = 25,675 'trips' ที่ย้าย แท้จริงต้องการรถกี่คัน? มีรถพอไหม?"

5. "Pipeline นี้เป็น batch หรือ streaming? ถ้า demand เปลี่ยน real-time จะทำยังไง?"

ตอบ: Batch processing — aggregate จาก historical data แล้วโหลดเป็น parquet, App ทำ what-if บนข้อมูลที่ pre-aggregated แล้ว

ถามตาม: "งั้นระบบนี้ใช้ตัดสินใจแบบ real-time ไม่ได้เลยเหรอ? ถ้าฝนตกทันที demand เพิ่ม 30% จะรู้ได้ยังไง?"

6. "ทำไม supply adjustment เป็นค่าตายตัว (เพิ่ม/ลบ X taxis) แต่ demand เป็น %? มันไม่ consistent ไหม?"

ตอบที่ควรเตรียม: Demand เป็น % เพราะเปลี่ยนตามปัจจัยภายนอก (สภาพอากาศ, event), Supply เป็นตัวเลขตายตัวเพราะจำนวนรถในระบบมีจำกัด (medallion)

ถามตาม: "แต่ถ้า supply = dropoff count ก็เปลี่ยนตาม demand อยู่แล้วนี่? เพิ่ม 1,000 taxis หมายความว่าอะไรจริงๆ?"

7. "การ aggregate เป็น zone × hour — ภายใน 1 ชม. มีการเปลี่ยนแปลง demand ไหม? Peak อยู่ที่นาที 0-10 กับ 50-60 ต่างกันไหม?"

ตอบที่ควรเตรียม: มี Wave Deployment แบ่ง 6 ช่วง × 10 นาที แต่ demand จริงเป็น hourly aggregate

ถามตาม: "แปลว่า Wave weights [25%, 25%, 20%, 15%, 10%, 5%] มาจากไหน? มีข้อมูลระดับนาทีจริงไหมที่ใช้ justify ตัวเลขพวกนี้?"

8. "Pipeline นี้มี data lineage อย่างไร? ถ้าผลลัพธ์ผิด จะ debug จากจุดไหน?"

ตอบที่ควรเตรียม: CSV → Delta (Cell 8) → Clean (Cell 12–14) → Spatial join (Cell 19) → Aggregate zone×hour → Export parquet → App

ถามตาม: "แต่ละขั้นมี validation ไหม? เช่น หลัง spatial join มี zone ไหนหายไปไหม? จำนวน rows เท่ากับก่อน join ไหม?"

9. "คุณบอกว่า MAPE 33% — แปลว่า forecast ผิดเฉลี่ย 1/3 ของค่าจริง แล้วนำไปคำนวณ cost-benefit ที่อ้างว่าได้ $500M-800M/ปี ได้อย่างไร?"

ตอบที่ควรเตรียม: พยากรณ์เป็น separate module ไม่ได้ feed เข้า allocation ตอนนี้ (เป็นข้อจำกัดที่ระบุไว้) — allocation ใช้ historical pattern ไม่ใช่ forecast

ถามตาม: "งั้นถ้าไม่ได้ใช้ forecast ประโยชน์จริงของการพยากรณ์คืออะไร? แค่ show case หรือมี application?"

10. "ถ้ามีข้อมูล GPS แท็กซี่แบบ real-time (รถว่างจริง) คุณจะเปลี่ยน pipeline อย่างไร?"

ตอบที่ควรเตรียม: เปลี่ยน supply จาก dropoff count → GPS status (available/occupied), เพิ่ม streaming layer (Kafka/Auto Loader), ลด aggregation interval จาก hour → minute

ถามตาม: "คุณมีข้อมูล GPS ไหม? ถ้าไม่มี จะหาจากไหนได้?"

คำถามด้านการเตรียมข้อมูล

นี่คือแนวคำถาม-คำตอบสำหรับการพรีเซนต์พรุ่งนี้ครับ เน้นขั้นตอนเตรียมข้อมูล (Cells 1–23):

📋 Q&A ขั้นตอนเตรียมข้อมูล

Q1: ข้อมูลมาจากไหน ขนาดเท่าไหร่

> ดาวน์โหลดจาก Kaggle dataset `gopalkalpnde/nyc-tlc-data` — เป็น NYC TLC Yellow Taxi trip records เดือนมกราคม ปี 2015–2018 รวม 4 ไฟล์ CSV ขนาด ~1.2 GB รวม 42,125,842 rows

---

Q2: ทำไมใช้แค่เดือนมกราคม

> เป็นข้อจำกัดจากแหล่งข้อมูลที่อาจารย์ให้มา — dataset มีเฉพาะเดือนมกราคม แต่เราระบุไว้ใน notebook และ README ว่าเป็น known limitation ถ้ามีข้อมูลทั้งปีจะสามารถจับ seasonality รายเดือนได้

Q3: ทำไมต้องแปลง CSV เป็น Delta

> สามเหตุผล: (1) Delta อ่านเร็วกว่า CSV เพราะมี columnar storage + statistics (2) Delta รองรับ ACID transactions และ time travel (3) บน Databricks การใช้ Delta บน UC Volume เป็น best practice สำหรับ data lakehouse

---

Q4: ข้อมูล 4 ปีมี schema ต่างกันยังไง แก้ยังไง

> 2015–2016 มี 20 คอลัมน์ (ใช้ GPS: `pickup_latitude/longitude`, `dropoff_latitude/longitude`) ส่วน 2017–2018 มี 18 คอลัมน์ (ใช้ `PULocationID/DOLocationID` แทน GPS)
>
> แก้โดยใช้ `unionByName(allowMissingColumns=True)` — คอลัมน์ที่ไม่มีในบางปีจะเป็น null อัตโนมัติ ผลลัพธ์ได้ 22 คอลัมน์

Q5: Missing values มีเท่าไหร่ จัดการยังไง ทำไมไม่ลบหมด

คอลัมน์	Missing	สาเหตุ
datetime, distance, fare	0%	—
GPS coords	43.84%	2017–2018 ใช้ LocationID แทน
LocationID	56.16%	2015–2016 ใช้ GPS แทน

> ถ้าลบ GPS null ทั้งหมด = ทิ้งข้อมูล 2017–2018 ทั้งหมด ถ้าลบ LocationID null = ทิ้ง 2015–2016 ทั้งหมด
>
> จึงใช้กลยุทธ์ **either-or**: เก็บแถวที่มี GPS ครบ **หรือ** LocationID ครบ อย่างน้อยชุดเดียว → ลบเพียง 0.66% (277,518 rows ที่ distance ≤ 0 หรือ fare ≤ 0)

Q6: สร้างฟีเจอร์อะไรขึ้นมาบ้าง

> **Time features** (Cell 14, 17):
>
> - `pickup_hour` (0–23) — จัดกลุ่มตามชั่วโมง หา rush hour
> - `pickup_dayofweek` (1=อาทิตย์…7=เสาร์) — แยก weekday/weekend
> - `is_weekend` (0/1) — เปรียบเทียบ pattern
> - `trip_duration_minutes` — คำนวณจาก dropoff - pickup
>
> **Spatial features** (Cell 19):
>
> - `pickup_borough`, `dropoff_borough` — map ทุก trip ไปยัง 1 ใน 6 boroughs

Q7: ทำไม 2015–2016 ต้อง spatial join ในขณะที่ 2017–2018 ใช้ broadcast join

2015–2016 มีแค่ GPS coordinates (lat/lon) ไม่มี LocationID → ต้องใช้ `st_contains(geom, st_point(lon, lat))` เพื่อหาว่าพิกัดอยู่ใน borough polygon ไหน

2017–2018 มี LocationID → ใช้ broadcast join ธรรมดา (lookup 263 rows ง่ายกว่า spatial join มาก)

ทั้งสองวิธีใช้ `broadcast()` เพื่อป้องกันการ shuffle ข้อมูล 23M+ rows ที่จะช้ามาก

Q8: Demand และ Supply คำนวณยังไง

Demand = จำนวน pickup ต่อ borough ต่อชั่วโมง — นับจาก `tpep_pickup_datetime` + `pickup_borough`

Supply = จำนวน dropoff ต่อ borough ต่อชั่วโมง — นับจาก `tpep_dropoff_datetime` + `dropoff_borough`

1 trip สร้าง 1 demand (ที่ zone ต้นทาง × ชม.ที่ขึ้น) และ 1 supply (ที่ zone ปลายทาง × ชม.ที่ลง) — นับที่คนละที่ คนละเวลา เลยทำให้ต่อ zone × hour แล้ว demand ≠ supply

---

Q9: Supply นับ dropoff แปลว่ารถว่างจริงเหรอ

ไม่ได้แปลว่ารถว่าง 100% — dropoff เป็น **proxy** (ตัวแทน) ของแท็กซี่ที่อยู่ในพื้นที่ ไม่ใช่จำนวนรถว่างจริง เพราะ:

- แท็กซี่มาส่งแล้วอาจไปเติมแก๊ส หรือเปลี่ยนกะ
- แท็กซี่ 1 คันทำหลาย trip/ชม. → นับซ้ำ (เป็น trips ไม่ใช่คันรถ)
- แต่ 25,675 trips ÷ ~2 trips/คัน/ชม. ≈ 12,800 คัน (ใกล้เคียง fleet จริง ~13,500 คัน) → proxy สมเหตุสมผล

Q10: ทำไมต้องกรอง trip_distance > 0 และ ≤ 100 ไมล์

`trip_distance = 0` = GPS ไม่ทำงานหรือระบบบันทึกผิด → ไม่ใช่ trip จริง

`> 100 ไมล์` = เกินขอบเขต NYC (กว้างสุด ~35 ไมล์) → น่าจะเป็นข้อมูลผิด

`fare_amount < 0` = ค่าโดยสารติดลบไม่มีจริง → ลบออก

Q11: ทำไมถึงมี NULL ใน borough หลัง spatial join

~1–2% ของ trip ที่ GPS อยู่นอก polygon ของ NYC zones — อาจเป็นจุดที่อยู่ในน้ำ นอกเขตเมือง หรือ GPS คลาดเคลื่อน ตอนหลังเติมด้วย "Unknown" และกรองออกจากการคำนวณ demand-supply

---

Q12: ขั้นตอนทั้งหมดเรียกว่าอะไร

Data Preprocessing & Feature Engineering — เป็นขั้นตอนเตรียมข้อมูลก่อนเข้าสู่การวิเคราะห์ ไม่ใช่ PCA (เป็นคนละสิ่ง) ไม่ใช่ Machine Learning — เป็น data pipeline ประกอบด้วย 4 ขั้น: (1) Data Ingestion (2) Data Cleaning (3) Feature Engineering (4) Metric Computation

---

Q13: ทำไมใช้ Spark ไม่ใช้ Pandas

ข้อมูล 42 ล้าน rows — Pandas โหลดเข้า memory ไม่ได้ (ต้องการ ~8–16 GB RAM) Spark กระจายการประมวลผลได้ และเราใช้ Delta on UC Volume ซึ่งเป็น distributed storage บน Databricks อยู่แล้ว

Q14: ใช้ข้อมูล 4 ปี แต่วิเคราะห์แค่ 2018 ทำไม

4 ปีใช้สำหรับ trend analysis (ดูแนวโน้ม demand ลดลง -31%) และ forecasting (SARIMA ต้องการหลายจุดข้อมูล)

ส่วน allocation analysis ใช้ 2018 เท่านั้นเพราะเป็นปีเดียวที่มี LocationID ครบสำหรับวิเคราะห์ระดับ zone (263 zones) และเป็นปีล่าสุดที่สะท้อนสถานการณ์ปัจจุบันมากที่สุด

Q15: ถ้าให้ทำใหม่จะปรับอะไร

3 อย่าง: (1) ข้อมูลทั้งปีไม่ใช่แค่มกราคม เพื่อจับ seasonality รายเดือน (2) เพิ่ม exogenous variables เช่น สภาพอากาศ วันหยุด อีเวนต์ จะได้ใช้ SARIMAX และ MAPE ลดลง (3) หา data source ที่บอกสถานะรถว่างจริง (GPS fleet tracking) แทนการใช้ dropoff count เป็น proxy

---

💡 ทิปส์ตอนพรีเซนต์

- ถ้าอาจารย์ถามที่ไม่ได้เตรียม → ตอบตรงๆ ว่า "เป็นข้อจำกัดที่ระบุไว้ในรายงาน" ดีกว่าเขียนคำตอบสด
- เน้น **either-or cleaning** เป็นจุดเด่น — แสดงว่าคิดแล้วไม่ลบข้อมูลมั่ว
- ถ้าถามเรื่อง PCA อีก → ย้ำว่า "งานนี้ไม่มี PCA เพราะไม่มีความจำเป็นต้องลดมิติ ตัวแปรที่สร้างขึ้นมีความหมายชัดเจนทั้งหมด"

คำถามด้านโมเดลพยากรณ์